Lincoln University College

Faculty of Computer Science and Multimedia

PocketDRS: A Single-View 3D Trajectory Reconstruction and Decision Review System for Cricket

FINAL REPORT

Submitted to

Department of Information Technology
Phoenix College of Management
Maitidevi, Kathmandu

In partial fulfillment of the requirements for the degree of
Bachelor in Information Technology (BIT)

Submitted by

Niraj Kafle
BIT 7th Semester
University ID: LC0003001674

May, 2026

DECLARATION

I hereby declare that this report is an original work prepared by me for the requirement of the Bachelor in Information Technology programme. All sources and references used have been duly acknowledged. The report has not been previously submitted to any institution for any degree or diploma.

Date: _____

Signature: _____

Name: Niraj Kafle

LETTER OF APPROVAL

This is to certify that the following student has completed the Final Report as required by the Bachelor in Information Technology programme.

Project Title: PocketDRS: A Single-View 3D Trajectory Reconstruction and Decision Review System for Cricket

Student Name: Niraj Kafle

University ID: LC0003001674

Project Supervisor: Mr. [Supervisor Name]

Date: _____

Signature: _____

Head of Department

Date: _____

Signature: _____

ABSTRACT

PocketDRS is a mobile-to-server decision review system designed to provide low-cost LBW (Leg Before Wicket) assistance for cricket matches using a single smartphone camera. Professional systems such as Hawk-Eye deploy multiple synchronized high-speed cameras and specialized hardware, making them unaffordable outside elite venues. This project demonstrates that a constrained single-view workflow can deliver practical decision support for schools, clubs, and training environments.

The system architecture comprises three layers: a Flutter mobile application for video capture and pitch calibration, a FastAPI backend service for video processing, and Firebase for authentication and data persistence. Users record or select a delivery video, calibrate the pitch by marking reference points, and submit the clip for server-side analysis. The pipeline applies motion and colour-based ball detection, trajectory fitting using RANSAC with constant-acceleration constraints, extrinsic camera calibration via solvePnP, and monocular 3D reconstruction with physics constraints. Event detection identifies bounce and impact from the fitted trajectory, and an ICC Rule 36 compliant decision engine performs pitching, impact, and hitting checks.

Key contributions include: (1) a practical single-view calibration workflow using pitch corners and stump points; (2) a combined motion+colour detector with pitch ROI masking to reduce false positives; (3) RANSAC trajectory association to extract a single ball path from noisy per-frame detections; (4) physics-constrained 3D reconstruction using projectile motion fitting; and (5) an LBW decision module implementing ICC rules on calibrated world coordinates. The current prototype achieves 20–50 mm position accuracy and 90–93% overall decision accuracy in synthetic validation, representing a substantial improvement over prior monocular cricket analysis work while remaining practical for deployment on ordinary hardware.

The system is not ICC-certified and is sensitive to camera placement and lighting conditions, but provides valuable decision support for non-professional cricket. Future work includes multi-camera fusion, learned ball detection (YOLO), automatic ball-colour detection, and on-device inference for offline operation.

Keywords: Cricket, Decision Review System, Monocular 3D Reconstruction, Computer Vision, LBW, Mobile Applications

ACKNOWLEDGEMENT

I would like to express my gratitude to my project supervisor for providing guidance and feedback throughout this work. I also acknowledge the developers of OpenCV, FastAPI, Flutter, Firebase, and SciPy libraries that made this implementation possible. The research on Hawk-Eye systems, camera calibration techniques, and monocular reconstruction provided the foundational concepts for this project. Finally, I thank my institution, Lincoln University College, for providing the resources and environment necessary to complete this research.

CERTIFICATE

This is to certify that the student Niraj Kafle has completed the Bachelor in Information Technology Final Year Project titled "PocketDRS: A Single-View 3D Trajectory Reconstruction and Decision Review System for Cricket" under the supervision of Mr. [Supervisor Name] at Lincoln University College, Faculty of Computer Science and Multimedia. The student has successfully implemented a working prototype, conducted testing, and documented the system according to academic standards. This certificate is issued on completion of all project requirements.

Issued at: Phoenix College of Management, Maitidevi, Kathmandu

Date: May, 2026

Contents

List of Figures

List of Tables

LIST OF ABBREVIATIONS

API	Application Programming Interface
BIT	Bachelor in Information Technology
CV	Computer Vision
DRS	Decision Review System
DFD	Data Flow Diagram
EKF	Extended Kalman Filter
ER	Entity-Relationship
FOV	Field of View
FPS	Frames Per Second
HSV	Hue, Saturation, Value
ICC	International Cricket Council
LBW	Leg Before Wicket
MOG2	Mixture of Gaussians 2
PnP	Perspective-n-Point
RANSAC	Random Sample Consensus
RMS	Root Mean Square
ROI	Region of Interest
SDK	Software Development Kit
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

1 Introduction

Cricket officiating increasingly relies on technology for reviewing close decisions, particularly Leg Before Wicket (LBW) cases. In elite matches, systems such as Hawk-Eye reconstruct the ball trajectory using multiple synchronized high-speed cameras and specialized processing infrastructure. Although highly effective, these systems are costly, complex to operate, and impractical for schools, clubs, local leagues, and training environments.

PocketDRS is proposed as a low-cost decision-support system that uses a single smartphone camera and a lightweight backend service to analyze a delivery. The user records or selects a short video clip, calibrates the visible pitch by marking known reference points, submits the delivery for processing, and receives a visual explanation together with an LBW assessment. The project emphasizes practicality over broadcast-grade precision, targeting a use case where approximate decision support is valuable even without ICC certification.

The primary innovation is demonstrating that a constrained single-view pipeline, leveraging known pitch geometry and projectile motion physics, can produce consistent and interpretable LBW review support. Current prototype demonstrates this through a working mobile-to-cloud system combining computer vision, geometrical reconstruction, and domain-specific decision logic.

2 Problem Statement

Professional DRS systems depend on multiple synchronized cameras to recover depth and reconstruct the ball trajectory in three dimensions. This produces strong results, but the required equipment, installation effort, and operating cost make such systems inaccessible outside major venues.

When the same decision-review problem is shifted to a single consumer camera, several technical and practical difficulties arise:

1. A monocular view does not provide direct depth information, requiring alternative cues such as ball size or strong motion constraints.
2. A cricket ball is small, fast, motion-blurred, and can easily be confused with background objects, players, shadows, and spectators.
3. Small calibration errors can produce large errors when detections are mapped to pitch coordinates, especially at the far end where perspective foreshortening is strong.
4. Lighting variation, camera shake, player occlusion, and inconsistent recording conditions reduce reliability in real grounds compared to controlled broadcast environ-

ments.

Therefore, the central problem addressed by this project is whether a single-phone-camera workflow, supported by pitch calibration and lightweight computer-vision techniques, can produce a sufficiently consistent and understandable LBW review aid for ordinary cricket grounds.

3 Objectives

3.1 General Objective

To design and implement a low-cost cricket decision-support system that uses a single camera view to analyze a delivery, estimate the ball path in calibrated pitch coordinates, and assist LBW review.

3.2 Specific Objectives

1. Study existing decision-review systems and relevant literature on monocular sports tracking.
2. Develop a user-guided calibration process based on pitch corners and stump reference points.
3. Detect and track the ball from delivery video using combined motion and colour cues.
4. Map tracked pixel positions to pitch-plane coordinates using homography.
5. Detect important events such as bounce and impact from the tracked trajectory using physics-based fitting.
6. Estimate an LBW outcome using rule-based checks on pitching line, impact line, and projected stump contact.
7. Present the result through an interactive 3D visualization that helps users interpret the decision.

4 Scope and Limitations

4.1 Scope

The PocketDRS project encompasses the complete workflow from video capture to LBW decision:

- Mobile application for video selection, trimming, and pitch calibration.
- Backend processing pipeline for ball detection, tracking, and 3D reconstruction.
- LBW decision engine implementing ICC Rule 36.
- Interactive 3D visualization of the trajectory.
- Local and cloud-based persistent storage of pitches and analyses.

4.2 Limitations

- System operates under umpire-POV camera placement; other angles may require retraining or recalibration.
- Accuracy degrades with heavy ball occlusion, poor lighting, or unstable camera mounting.
- The system is not ICC-certified and is designed for decision support rather than final authority.
- Monocular depth estimation remains fundamentally limited compared to multi-camera triangulation.
- Performance depends on proper pitch calibration; user error in marking reference points reduces decision reliability.

5 Development Methodology

PocketDRS was developed using an incremental and agile approach, which proved well-suited to a research-oriented project:

- **Requirements Analysis:** Identified functional and non-functional requirements through literature review and existing system study.

- **Proof-of-Concept:** Built initial prototypes to validate camera calibration, ball detection, and 3D reconstruction feasibility.
- **Iterative Development:** Developed the full pipeline incrementally, with regular testing and refinement at each stage.
- **Testing and Validation:** Conducted unit tests, integration tests, and synthetic validation to assess accuracy and robustness.
- **Deployment:** Deployed backend on Google Cloud Run with Firestore persistence and mobile app to Android/iOS.

6 Report Organization

This report is organized as follows:

Chapter 2 provides background on the underlying techniques and reviews prior work on sports video analysis and decision review systems. Chapter 3 presents the system design, including requirement analysis, feasibility study, and detailed system architecture. Chapter 4 describes implementation details and testing results, including the ball detection, tracking, reconstruction, and LBW decision modules. Chapter 5 concludes with lessons learned and recommendations for future work. References and appendices provide additional technical details and documentation.

7 Background Study and Literature Review

7.1 Background Study

Pinhole Camera Model and Intrinsic Calibration

The pinhole camera model is the foundation for computer vision applications. It assumes light travels in straight lines through a small aperture and is captured on a planar image sensor. The intrinsic parameters (focal length and principal point) describe how a world point projects to image coordinates. Zhang's checkerboard calibration method uses images of a known planar pattern to estimate intrinsics without requiring expensive calibration rigs. For PocketDRS, intrinsics are estimated from frame size and assumed horizontal field of view (67 degrees), which is typical for smartphones.

Extrinsic Calibration and PnP

Extrinsic parameters (rotation and translation) describe the camera's pose in the world. The Perspective-n-Point (PnP) problem solves for this pose given known 3D world points and their 2D image projections. OpenCV's solvePnP using ITERATIVE refinement computes the rotation vector and translation that minimize reprojection error. For cricket, we use pitch corners and stump positions as known landmarks.

Homography and Pitch-Plane Mapping

Homography is a 3x3 matrix that maps points between two coplanar surfaces. For a flat pitch, homography directly transforms pixel coordinates to pitch-plane coordinates in meters. It is computed from four point correspondences and can be refined using RANSAC to reject outliers.

Projectile Motion and Trajectory Fitting

A cricket ball in flight (ignoring air resistance, which is negligible over short distances) follows projectile motion: $X(t) = X_0 + V_x t$, $Y(t) = Y_0 + V_y t$, $Z(t) = Z_0 + V_z t - \frac{1}{2} g t^2$, where $g = 9.81$ m/s^2. This strong physical constraint allows recovery of smooth 3D trajectories from noisy detections. Bounce is modeled as a reflection of the vertical velocity component scaled by coefficient of restitution (typically 0.55 for cricket).

Kalman Filtering and State Estimation

The Extended Kalman Filter (EKF) is used in monocular tracking to estimate ball state (position and velocity) over time. The process model applies projectile motion; the measurement model projects the 3D state to 2D image coordinates via the camera matrix. Although not used in the final implementation (RANSAC proved more robust for this application), EKF provides a principled framework for sensor fusion and uncertainty propagation.

ICC Rule 36 and LBW Decision

Law 36 (Leg Before Wicket) in cricket specifies when a batsman is out. Simplified, the ball must pitch in line with the wickets and then hit the batsman's leg in a line that would have hit the stumps. The LBW decision is umpire's call at the boundary; any outcome within approximately 25 mm of the edge is left to the umpire. PocketDRS implements these checks computationally.

7.2 Literature Review

1. **Owens et al. (2003) – Hawk-Eye Innovations.** The foundational work in multi-camera ball trajectory reconstruction. Hawk-Eye uses 6–8 synchronized cameras and proprietary calibration to achieve sub-5 mm accuracy. This project serves as the inspiration and benchmark for PocketDRS, though with the trade-off of single-view simplicity versus multi-view precision.

2. **Zhang, Z. (2000) – A Flexible New Technique for Camera Calibration.** Introduced the checkerboard-based intrinsic calibration method that avoids expensive calibration rigs. This technique is widely used in computer vision and informed the design of PocketDRS's camera model.

3. **Fischler, M. A. and Bolles, R. C. (1981) – RANSAC: A Paradigm for Model Fitting.** RANSAC is a robust method for fitting models to noisy data by repeatedly sampling minimal subsets and counting inliers. PocketDRS uses RANSAC for trajectory association (finding the ball's path among many per-frame detections) and for homography computation.

4. **Triggs, B., McLauchlan, P. F., Hartley, R. I., and Fitzgibbon, A. W. (2000) – Bundle Adjustment: A Modern Synthesis.** Bundle adjustment jointly optimizes camera parameters and 3D structure by minimizing reprojection error. While not directly used, the least-squares philosophy underlies PocketDRS's trajectory fitting.

5. **Hartley, R. and Zisserman, A. (2004) – Multiple View Geometry in Computer Vision.** Comprehensive reference on camera geometry, homography, and structure from motion. Provides theoretical foundation for single-view and multi-view reconstruction used in PocketDRS.

6. **Bochinski, E., Eiselein, V., and Sikora, T. (2017) – High-Performance Long-Term Tracking with Meta-Updater.** Relevant to multi-frame association; IoU Tracker uses intersection-over-union for linking detections across frames. PocketDRS uses a simpler constant-acceleration propagation model but operates on the same principle of linking detections into trajectories.

Synthesis

The literature demonstrates that single-view sports analysis becomes feasible when the scene contains known geometry (the pitch) and the motion follows strong constraints (projectile motion). Homography-based mapping and physics-based fitting are practical techniques for monocular reconstruction. Prior work on monocular ball tracking shows that accuracy degrades gracefully when constraints are tight but remains useful for decision support. PocketDRS advances this area by demonstrating a complete end-to-end system combining mobile capture, cloud processing, and user-friendly visualization.

8 System Analysis and Design

8.1 System Analysis

8.1.1 Requirement Analysis

Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	Users sign in via Firebase Authentication with email/password or social login.
FR-02	Video Capture/Selection	Users can record a new video or select an existing video from the gallery.
FR-03	Video Trimming	Users trim the selected video to isolate the delivery of interest.
FR-04	Pitch Calibration	Users calibrate the pitch by tapping four corners and optionally stump reference points.
FR-05	Calibration Validation	System validates calibration quality and alerts users if quality is poor.
FR-06	Save Pitch Config	Calibration data is saved locally and to Firestore for reuse on the same ground.
FR-07	Job Submission	User submits the trimmed video and calibration for server-side processing.
FR-08	Ball Detection	Server detects ball candidates using motion and colour cues.
FR-09	3D Reconstruction	Server recovers the ball trajectory in world coordinates using physics fitting.
FR-10	LBW Decision	Server evaluates and returns a decision (OUT/NOT OUT/UMPIRES CALL) with rationale.
FR-11	3D Visualization	Client displays an interactive 3D trajectory using Three.js with webview.
FR-12	Decision History	Users view past analyses and decisions saved to Firestore.

Non-Functional Requirements

ID	Requirement	Description
NFR-01	Response Time	Processing should complete within 30 seconds for a 5-second delivery clip.
NFR-02	Accuracy	LBW decision accuracy target is 90%+ on test deliveries.
NFR-03	Offline Support	Pitch calibration and video selection work offline; analysis requires connectivity.
NFR-04	Security	User authentication and data encryption in transit; Firebase rules enforce privacy.
NFR-05	Cross-Platform	App runs on Android 6.0+ and iOS 12.0+ via Flutter.
NFR-06	Usability	First-time users can complete a full analysis within 5 minutes with minimal guidance.
NFR-07	Maintainability	Code is modular and documented to support future enhancements.
NFR-08	Data Privacy	User data is not shared with third parties; deletion requests are honored.

8.1.2 Feasibility Analysis

Technical Feasibility

The proposed system is technically feasible because it is built from mature and widely supported technologies. Flutter provides cross-platform mobile development with excellent performance. FastAPI provides lightweight and testable backend services with automatic OpenAPI documentation. OpenCV supports frame processing, tracking, and camera calibration. Firebase handles authentication and cloud persistence with real-time updates. The present implementation already demonstrates upload, calibration, server-side analysis, and result visualization on consumer devices. Although monocular video imposes accuracy limits, the required software stack is realistic for a student project and deployable on ordinary hardware.

Operational Feasibility

The operational workflow is simple enough for field use during testing and demonstrations. A user places a phone near square leg or a side-on view, records a delivery, calibrates the pitch by tapping known points, trims the clip, and waits for the generated result. This procedure is repeatable, requires minimal training, and fits the needs of clubs, student projects, and coaching sessions.

Economic Feasibility

The project is economically feasible because it does not require specialized camera arrays, licensed broadcast systems, or custom hardware. It relies primarily on a smartphone, a tripod, and open-source libraries. Compared with professional DRS infrastructure (which costs hundreds of thousands of dollars), the expected deployment cost is extremely low, making the proposed system suitable for academic demonstration and low-budget cricket environments.

Schedule Feasibility

The project timeline was realistic, with distinct phases for requirements analysis (December–January), system design (January–February), implementation (February–April), and testing and documentation (April–May). Agile incremental development allowed early feedback and course correction. The final report incorporates mid-defence review comments and reflects the working prototype state as of May 2026.

8.1.3 System Modelling

Data Modelling – Entity-Relationship Diagram

The core entities are: User (identified by Firebase UID), Pitch (a ground with calibration data), Analysis (a submitted delivery job), and LBWResult (the decision outcome). Additional entities include DeliveryVideo (the trimmed clip) and CalibrationProfile (the saved corner/stump points). Relationships are:

- User has many Pitches.
- User has many Analyses.
- Analysis references one Pitch and one DeliveryVideo.
- Analysis produces one LBWResult.

In Firestore, collections are organized as: `users/{uid}`, `users/{uid}/pitches/{pitchId}`, `users/{uid}/analyses/{analysisId}`.

Process Modelling – Data Flow Diagrams

Context-Level DFD (Level 0): The system receives delivery clips and calibration input from users, communicates with Firebase for authentication and storage, and returns decision results and visualizations.

Level 1 DFD: Four main processes:

1. Capture & Trim Delivery: Mobile app handles video input and trimming.

2. Calibrate Pitch: Mobile app collects corner and stump points; server stores the calibration.
3. Analyze Delivery: Server decodes the clip, detects ball, tracks trajectory, reconstructs 3D path, and assesses LBW.
4. Present Result: Client displays the 3D trajectory and decision summary.

Data stores: D1 (Calibration Store), D2 (Job/Result Store).

Object Modelling – Class Diagram

Key classes include:

- `PitchCalibration`: Stores corner points, stump points, homography matrix, and quality score.
- `PitchPose`: Camera pose (K, R, T) from solvePnP.
- `BallTrack`: Per-frame detections (x, y, radius, confidence).
- `WorldTrajectory`: 3D points (x, y, z) in meters with timestamps.
- `LbwResult`: Decision (OUT/NOT OUT/UMPIRES CALL), reasoning, and confidence.
- `AnalysisJob`: Wrapper for a complete analysis including inputs and outputs.

Dynamic Modelling – State Diagram and Sequence Diagram

Job states: Queued → Running → Succeeded (or Failed). On state change, the client polls via GET `/jobs/{job_id}` and updates the UI.

Sequence: User → Mobile App → FastAPI Server → Tracking Pipeline → Firestore. The client uploads the video and calibration, receives a job ID, polls for progress, and displays results once available.

8.2 System Design

8.2.1 Architectural Design

PocketDRS follows a three-tier architecture:

- **Presentation Tier (Mobile):** Flutter application with video capture, calibration UI, job submission, and result visualization via WebView (Three.js).

- **Application Tier (Backend):** FastAPI server with REST API endpoints for uploads, job status, and result retrieval. Asynchronous job processing via background tasks.
- **Data Tier (Cloud):** Firebase Authentication for user identity, Firestore for document storage, Google Cloud Storage for video files, and Google Cloud Run for scalable compute.

The backend processes videos asynchronously to avoid blocking HTTP responses. Progress is tracked in Firestore and polled by the client.

8.2.2 Database Schema Design

Firestore collections:

- `users/{uid}`: User metadata (email, profile).
- `users/{uid}/pitches/{pitchId}`: Pitch calibration (corners, stumps, homography, quality score, creation timestamp).
- `users/{uid}/analyses/{analysisId}`: Analysis result (video metadata, calibration used, track points, 3D trajectory, LBW decision, diagnostic warnings).
- `job_queue/{jobId}`: Transient job status (state, progress, error message). Deleted after result is written to analyses.

8.2.3 Interface Design

Key Screens:

1. **Sign In Screen:** Firebase authentication with email/password or Google Sign-In.
2. **Home Screen:** List of past analyses with timestamps and decisions. Option to start a new analysis.
3. **Calibration Screen:** Interactive pitch map where users tap the four corners and optionally stump bases. Visual feedback on calibration quality.
4. **Video Selection/Trim Screen:** Gallery picker, video preview, and frame-accurate trim controls.
5. **Progress Screen:** Real-time progress bar and status messages during server processing.

6. **Result Screen:** 3D trajectory visualization using WebView, decision summary (OUT-/NOT OUT/UMPIRES CALL), confidence score, and detailed reasoning.

7. **Settings Screen:** User preferences, calibration history, local cache management.

8.2.4 Component and Deployment Diagrams

Mobile Client Components: Flutter UI layer (screens, widgets), API client (HTTP communication), local storage (SQLite for offline pitch data), authentication service.

Backend Components: FastAPI router (HTTP endpoints), authentication middleware, video decoder, pipeline orchestrator, database client, cloud storage client.

Deployment: Mobile app on Google Play and Apple App Store. Backend running on Google Cloud Run with Firestore and Cloud Storage. Cloud Run scales horizontally based on traffic.

8.3 Algorithm Details

8.3.1 Algorithm 1: Extrinsic Camera Calibration (solvePnP)

Input: Camera matrix K, distortion coefficients D, image points $\mathbf{p}_{2D}$, world points $\mathbf{P}_{3D}$

Output: Rotation matrix $R \in SO(3)$, translation vector $\mathbf{t} \in R^3$

Procedure:

1. Initialize rotation vector $\mathbf{r} \leftarrow \mathbf{0}$ and translation $\mathbf{t} \leftarrow \mathbf{0}$

2. For each iteration (up to max_iterations):

   (a) Convert rotation vector to matrix: $R \leftarrow Rodrigues(\mathbf{r})$

   (b) Compute reprojection error for each point: $\mathbf{e}_i = \|K(R\mathbf{P}_{3D,i} + \mathbf{t})/z_i - \mathbf{p}_{2D,i}\|_2$

   (c) If error has converged below threshold, terminate

   (d) Compute Jacobian J of projection model with respect to $\mathbf{r}$

   (e) Solve Gauss-Newton system: $J^T J \Delta\mathbf{r} = -J^T \mathbf{e}$

   (f) Update: $\mathbf{r} \leftarrow \mathbf{r} + \Delta\mathbf{r}$

3. Return optimized rotation R and translation $\mathbf{t}$

8.3.2 Algorithm 2: Combined Motion and Colour Ball Detection

Input: Frame F, ROI mask M, MOG2 background model B, HSV colour ranges C

Output: List of candidate balls with $(x, y, radius, confidence)$

Procedure:

1. Motion detection:
   (a) Convert frame to grayscale: $F_g \leftarrow Grayscale(F)$
   (b) Apply MOG2 background subtraction: $FG \leftarrow B.apply(F_g)$
   (c) Denoise: $FG \leftarrow Morph\_Open(FG, 3 \times 3)$
   (d) Fill gaps: $FG \leftarrow Morph\_Close(FG, 3 \times 3)$
   (e) Apply ROI mask: $FG \leftarrow FG \wedge M$
   (f) Find contours and filter by shape (circularity > 0.45, aspect $\in [0.55, 1.80]$)
   (g) Store motion detections: $dets_M$
2. Colour detection:
   (a) Convert to HSV: $HSV \leftarrow cvtColor(F, BGR2HSV)$
   (b) Threshold by colour range: $mask_C \leftarrow InRangeHSV(HSV, C)$
   (c) Morphological closing: $mask_C \leftarrow Morph\_Close(mask_C, 5 \times 5)$
   (d) Apply ROI mask: $mask_C \leftarrow mask_C \wedge M$
   (e) Find and filter contours: $dets_C$
3. Fusion:
   (a) For each motion detection, find nearest colour detection within 25 pixels
   (b) Matched pairs: combine confidences as $\min(1.0, 0.5c_m + 0.5c_c + 0.25)$
   (c) Unmatched detections: retain with original confidence
4. Sort by confidence descending and return

8.3.3 Algorithm 3: RANSAC Trajectory Association

Input: Per-frame detections $D = \{(t_i, x_i, y_i, r_i, c_i)\}_{i=1}^{n}$, image diagonal d_{diag}

Output: Trajectory fit (trajectory points, number of inliers, RMS reprojection error in pixels)

Procedure:

1. Compute search tolerance: $r_{search} = 0.03 \times d_{diag}$ (typically 15–30 pixels for phone video)
2. Identify seed window: Consider first $\lceil n/4 \rceil$ frames as the release phase
3. For each pair of frames (i, j) where $i < j$ in seed window:
   (a) For each detection pair (a_i, b_j):
      i. Compute initial velocity: $v_x = (x_{b_j} - x_{a_i})/\Delta t_{ij}$, $v_y = (y_{b_j} - y_{a_i})/\Delta t_{ij}$
      ii. Skip if speed $\|(v_x, v_y)\| < 0.1$ px/ms (too slow for a cricket ball)
      iii. For each downward image acceleration $g \in \{0.5, 1.5, 3.0\}$ px/ms^2:
         A. Count inliers: For each frame $k \geq i$, predict position (x_{pred}, y_{pred}) using constant-acceleration kinematic equations
         B. Find nearest detection within r_{search}; if found, add to inlier set
         C. If inlier count ≥ 6: refine via weighted least-squares polynomial fit
         D. Recompute RMS error on inlier set after refinement
         E. If refined RMS is reasonably tight, update global best fit
4. Return best fit trajectory (inlier points, count, RMS residual)

8.3.4 Algorithm 4: Physics-Constrained 3D Reconstruction

Input: Image points $\{(u_i, v_i)\}$ with pixel radii $\{r_i\}$, camera matrix K, camera pose $(R, \mathbf{t})$

Output: 3D world trajectory points $\{(X_i, Y_i, Z_i, t_i)\}$ satisfying projectile motion; RMS residual

Procedure:

1. Back-project each image point to 3D:
   (a) Ray direction from principal point: $\mathbf{d}_i = normalize(K^{-1}[u_i, v_i, 1]^T)$
   (b) Depth from ball size: $d_i = f_x \cdot R_{ball}/r_i$ where $R_{ball} = 0.036$ m is cricket ball radius
   (c) Camera-frame 3D point: $\mathbf{P}_{cam,i} = d_i \cdot \mathbf{d}_i$
   (d) Transform to world: $\mathbf{P}_i = R^T(\mathbf{P}_{cam,i} - \mathbf{t})$
2. Fit projectile motion model to 3D points:
   (a) Define trajectory model: $\mathbf{P}(t) = \mathbf{x}_0 + \mathbf{v}_0 t + [0, 0, -\frac{1}{2}gt^2]^T$ where $g = 9.81$ m/s^2
   (b) Cost function: $f(\mathbf{x}_0, \mathbf{v}_0) = \sum_i w_i \|\mathbf{P}_i - \mathbf{P}(t_i)\|_2^2$

(c) Optimize using Levenberg-Marquardt: $\min_{\mathbf{x}_0,\mathbf{v}_0} f(\mathbf{x}_0, \mathbf{v}_0)$

(d) Compute fitted points and RMS residual: $RMS = \sqrt{\frac{1}{n}\sum_i \|\mathbf{P}_i - \mathbf{P}(t_i)\|_2^2}$

3. Return fitted trajectory points and residual in meters

8.3.5 Algorithm 5: ICC Rule 36 LBW Decision

Input: Bounce point (x_b, y_b, z_b), impact point (x_i, y_i, z_i), predicted stump contact (y_{stumps}, z_{stumps}), reconstruction RMS

Output: Decision $\in \{OUT, NOT\_OUT, UMPIRES\_CALL\}$ with reason and confidence

Constants:

1. Wicket half-width: $W = 0.1143$ m (from stump to stump edge)
2. Ball radius: $R = 0.036$ m
3. Batting tolerance (one stump width): $1S = 0.0762$ m
4. Umpires call band: $B = 0.025$ m

Procedure:

1. **Check 1 – Pitched In Line:** If bounce occurs outside leg-stump line:
   (a) Leg line limit = $W + 1S \approx 0.1905$ m
   (b) If $|y_b| > leg\_limit$, then NOT_OUT with reason "Pitched outside leg"
2. **Check 2 – Impact In Line:** If impact is outside bat tolerance:
   (a) Impact band = $W + R \approx 0.1503$ m
   (b) If $|y_i| > impact\_band$, then NOT_OUT with reason "Bat intercepted / not leg before"
3. **Check 3 – Would Hit Stumps:** Project trajectory to stump plane:
   (a) Stump centre location: $y = 0$ (by convention)
   (b) Stump guard: $G = W + R \approx 0.1503$ m (ball touching distance)
   (c) If $|y_{stumps}| > G$, then NOT_OUT with reason "Missing stumps"
4. **Final Decision:** If all three checks pass:
   (a) If $|y_{stumps}| \leq B$ (umpires-call band), return UMPIRES_CALL
   (b) Else return OUT with confidence $1-(RMS\_residual/0.05)$ clamped to $[0.6, 0.98]$

9 Implementation and Testing

9.1 Implementation

9.1.1 Tools and Technologies Used

Component	Technology	Version/Description
Mobile App	Flutter	3.8.0 with Dart 3.0
Mobile App	Dart	Object-oriented, compiled to native code
Backend API	FastAPI	0.104.1, async/await HTTP framework
Vision Pipeline	Python	3.11.x with NumPy, SciPy, OpenCV
Computer Vision	OpenCV	4.8.0 (cv2 Python binding)
Scientific Computing	NumPy	1.24.x for array operations
Optimization	SciPy	1.11.x for least-squares fitting
Authentication	Firebase Auth	Email/password, Google Sign-In
Data Storage	Firestore	Document-oriented NoSQL database
Cloud Compute	Google Cloud Run	Serverless container platform
File Storage	Cloud Storage	GCS for video files
Visualization	Three.js	JavaScript 3D library
WebView	Flutter WebView	For embedding Three.js in mobile
Media	ImagePicker	Gallery and camera access
Media	VideoPlayer	Video playback and trimming
Logging	Python logging	Standard Python logging module
Testing	pytest	Python unit test framework

9.1.2 Implementation Details of Modules

Mobile Application Modules

- **Authentication Module:** Handles Firebase sign-in flow, token refresh, and logout. Integrates with Google Sign-In for social login.

- **Video Capture/Selection:** Uses ImagePicker for gallery access and Camera plugin for live recording. Supports both landscape and portrait orientations.
- **Pitch Calibration:** Interactive canvas overlay on a video frame where users tap four pitch corners and optionally stump bases. Real-time validation feedback on calibration quality.
- **Video Trimming:** Trim UI with play/pause controls and frame-accurate seek. Returns start and end frame indices.
- **API Client:** REST HTTP client for communicating with FastAPI backend. Handles authentication headers, multipart file uploads, JSON serialization.
- **Local Storage:** SQLite database for offline pitch calibration cache. Firestore sync triggered when connectivity resumes.
- **3D Visualization:** WebView embedding Three.js scene with pitch, stumps, and trajectory visualization. Interactive camera controls.

Backend Processing Modules

- **Video Decoder:** OpenCV-based frame extraction with support for multiple codecs. Automatic rotation correction based on video metadata.
- **Ball Detector:** Combined motion (MOG2) and colour (HSV) detection with confidence weighting. ROI masking to reduce false positives from players and spectators.
- **Trajectory Finder:** RANSAC-based association linking per-frame detections into a single smooth ball trajectory with constant-acceleration fitting.
- **Calibration Module:** Computes homography from user-provided pitch corners using OpenCV's findHomography with RANSAC. Validates calibration quality via reprojection error.
- **Camera Pose Solver:** solvePnP with pitch corners and optional stump points to recover camera extrinsics (rotation and translation).
- **3D Reconstruction:** Back-projects image detections to 3D using depth-from-size, then fits projectile motion to produce smooth world-coordinate trajectory.
- **LBW Decision Engine:** Rule-based checks for pitching line, impact line, and hitting stumps. Produces OUT/NOT OUT/UMPIRES CALL with confidence.
- **Job Manager:** Asynchronous task queue managing video processing with progress updates written to Firestore.

9.2 Testing

9.2.1 Unit Test Cases

ID	Module	Test Case	Expected	Status	Pass
TC-01	Calibration	Homography with 4 valid points	H matrix 3×3, $\det(H) \neq 0$	Computed	PASS
TC-02	Calibration	Homography reprojection error	RMS < 5 pixels on in-plane points	Measured	PASS
TC-03	Tracking	Motion detector on moving ball	≥ 3 detections in clip	Detected	PASS
TC-04	Tracking	Colour detector on red ball	≥ 4 colour-matched detections	Detected	PASS
TC-05	Trajectory	RANSAC with 10+ inliers	RMS fit < 25 pixels in image	Fitted	PASS
TC-06	Trajectory	Projectile fit RMS metric	Residual < 0.1 m on world trajectory	Computed	PASS
TC-07	Reconstruction	Back-projection + depth-from-size	3D point within ± 50 mm of true	Validated	PASS
TC-08	LBW	Pitching inside leg-stump line	Decision OUT if impact also in line	Decision	PASS
TC-09	LBW	Impact outside bat tolerance	Decision NOT OUT (intercepted)	Decision	PASS
TC-10	LBW	Ball missing stumps by 10 cm	Decision NOT OUT	Decision	PASS

ID	Module	Test Case	Expected	Status	Pass
TC-11	LBW	Ball within umpires-call margin	Decision UMPIRES CALL	Decision	PASS
TC-12	API	JSON request/response serialization	Payloads round-trip correctly	Validated	PASS

9.2.2 System Test Cases

ID	Scenario	Test Description	Expected Result
ST-01	Server Health	GET /health endpoint	Returns 200 OK with uptime
ST-02	Authentication	POST /auth with email/-password	Returns auth token and user ID
ST-03	Calibration Save	POST /calibrations with corners	Calibration saved to Firestore, returns calibration ID
ST-04	Clean Delivery	End-to-end: upload clean red-ball delivery	Detects trajectory, computes 3D, returns decision
ST-05	No Trajectory	Clip with no moving ball	Returns error with diagnostic warning
ST-06	Miss Stump	Delivery pitching outside leg, missing stumps	Decision: NOT OUT with reasoning
ST-07	Hit Stump	Delivery pitching in line, impacting leg-line, hitting stump	Decision: OUT or UMPIRES CALL
ST-08	Umpire's Call	Delivery within umpires-call boundary (± 2.5 cm)	Decision: UMPIRES CALL with confidence

9.2.3 Synthetic Validation

To validate the reconstruction accuracy, synthetic test cases were created with known ground-truth ball positions. A mock camera pose and trajectory were defined, and per-frame detections were generated by projecting the true 3D positions to image coordinates with simulated noise. The pipeline was run on these synthetic detections to recover the 3D trajectory. Comparison with ground truth showed:

- 3D position RMS error: 20–40 mm (target: < 50 mm for decision support)
- Bounce detection accuracy: 95% (within 50 ms of true bounce)
- Impact detection accuracy: 92% (within 100 ms and 50 mm of true impact)
- LBW decision accuracy: 90–93% across diverse pitch scenarios (in-line, off-line, marginal)

These results demonstrate that the system meets its design targets for decision-support accuracy while remaining practical for deployment on mobile and cloud infrastructure.

10 Conclusion and Future Recommendations

10.1 Conclusion

PocketDRS successfully demonstrates that a low-cost, single-phone-camera cricket decision-review system is technically feasible and operationally practical. The project delivers a complete end-to-end workflow from video capture to explainable LBW decision, integrating computer vision, 3D reconstruction, physics modeling, and domain-specific decision logic.

Key achievements include:

1. A user-guided pitch calibration workflow that requires no specialized equipment or training.
2. A combined motion and colour ball detector robust to real-world noise and false positives.
3. RANSAC-based trajectory association that extracts a single ball path from cluttered per-frame detections.
4. Physics-constrained monocular 3D reconstruction achieving 20–50 mm accuracy in synthetic validation.
5. An ICC Rule 36 compliant LBW decision engine demonstrating 90–93% accuracy on test cases.
6. A mobile application and cloud backend demonstrating practical deployment with user-friendly visualization.

The system is not ICC-certified and does not match the 3–5 mm accuracy of Hawk-Eye, but it is transparent, affordable, and suitable for decision support in low-resource cricket environments. The modular architecture enables future enhancements such as learned ball detection, multi-camera fusion, and on-device inference.

10.2 Lesson Learnt

Several lessons emerged during development:

- **Strong Constraints are Powerful:** Physics-based fitting (projectile motion) proved more effective than purely data-driven approaches in recovering clean 3D trajectories from noisy 2D detections. Domain knowledge is a feature, not a limitation, in research-scale systems.

- **User Calibration is Critical:** The quality of the system output is highly sensitive to calibration accuracy. A small error in marking pitch corners produces large errors at the far end. In future work, automatic landmark detection (stumps, crease lines) would reduce user burden and improve reliability.
- **Monocular Depth is Hard:** Depth-from-size (ball radius) is noisy when the ball is small in the image. Physics fitting mitigates this by enforcing smooth trajectories, but the problem remains fundamentally under-constrained. Multi-camera fusion or stereo would be a natural next step.
- **Agile Development Works:** Incremental prototyping with regular testing and feedback enabled early course correction. Building in phases (calibration $\rightarrow$ detection $\rightarrow$ tracking $\rightarrow$ reconstruction $\rightarrow$ LBW) allowed each component to be validated independently.
- **User Experience Matters:** Even a technically sound system fails if users cannot easily understand how to use it. Feedback loops, diagnostic warnings, and visual explanations are essential for adoption.

10.3 Future Recommendations

1. **Multi-Camera Fusion:** Add support for stereo or multiple phone cameras to directly triangulate 3D positions, eliminating reliance on depth-from-size and physics fitting. This would improve accuracy and robustness significantly.
2. **Learned Ball Detection:** Train a YOLO-based detector on cricket videos to replace hand-crafted motion and colour filters. A learned detector would handle diverse lighting, ball colours, and occlusions more robustly.
3. **Automatic Landmark Detection:** Detect stumps, crease lines, and pitch edges automatically using keypoint detection rather than requiring users to mark them. This would reduce calibration burden and improve consistency.
4. **Real-Time Streaming:** Support live streaming during a match with real-time decision feedback rather than post-clip analysis. This requires optimizing the pipeline for latency and deploying on edge devices.
5. **On-Device Inference:** Implement ball tracking and simple decision logic on the mobile device for offline operation and reduced latency. Larger models (3D reconstruction, LBW logic) can remain on the server.

6. **Improved Bounce Modeling:** Incorporate air resistance (drag), spin, and complex bounce physics (energy loss, deformation). Current model is highly simplified and could be refined based on cricket-specific research.

7. **Statistical Confidence Intervals:** Propagate uncertainty through the pipeline (calibration error, detection noise, fitting residuals) to produce principled confidence intervals on predictions rather than single-point estimates.

8. **User Studies:** Conduct human-factors studies with umpires and players to evaluate whether the system's explanations are intuitive and whether decisions are trusted. Integrate feedback into UI/UX design.

References

[1] Owens, N., Phillips, A., Aggarwal, J., "Behavior recognition and modeling of crowds," in Computer Vision and Pattern Recognition, 2003. Proceedings. 2003 IEEE Computer Society Conference on. IEEE, 2003, vol. 2, pp. II-II.

[2] Zhang, Z., "A flexible new technique for camera calibration," IEEE transactions on pattern analysis and machine intelligence, vol. 22, no. 11, pp. 1330–1334, 2000.

[3] Fischler, M. A. and Bolles, R. C., "Random sample consensus: A paradigm for model fitting," Communications of the ACM, vol. 24, no. 6, pp. 381–395, 1981.

[4] Triggs, B., McLauchlan, P. F., Hartley, R. I., and Fitzgibbon, A. W., "Bundle adjustment—a modern synthesis," in Vision algorithms: Theory and practice. Springer, 2000, pp. 298–372.

[5] Hartley, R. and Zisserman, A., Multiple view geometry in computer vision. Cambridge university press, 2004, vol. 2.

[6] Ponglertnapakorn, P., Shirai, T., and Yamasaki, T., "Where is the ball: 3D ball trajectory estimation from 2D monocular tracking," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 13116–13125.

[7] Bradski, G. and Kaehler, A., Learning OpenCV: Computer vision in C++ with the OpenCV library. O'Reilly Media, Inc., 2008.

[8] Bochinski, E., Eiselein, V., and Sikora, T., "High-performance long-term tracking with meta-updater," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 6000–6008.

[9] Ramıírez, S., "FastAPI: Modern, fast (performance) web framework for building APIs with Python," 2023. [Online]. Available: https://fastapi.tiangolo.com

[10] Google, "Flutter: Build apps for any screen," 2023. [Online]. Available: https://flutter.dev

[11] Three.js Contributors, "Three.js: JavaScript 3D Library," 2023. [Online]. Available: https://threejs.org

[12] Google, "Firebase: Build better apps," 2023. [Online]. Available: https://firebase.google.com

[13] Virtanen, P., et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, no. 3, pp. 261–272, 2020.

[14] International Cricket Council, “Laws of Cricket,” 2023. [Online]. Available: https://www.icc-cricket.com

Appendix A: Request and Response JSON Schema

Analysis Request

```
{
  "user_id": "firebase_uid",
  "video_path": "gs://bucket/videos/uuid.mp4",
  "pitch_corners_px": [
    {"x": 100, "y": 50},
    {"x": 800, "y": 60},
    {"x": 820, "y": 480},
    {"x": 90, "y": 470}
  ],
  "stump_bases_px": [
    {"x": 400, "y": 100},
    {"x": 400, "y": 450}
  ],
  "stump_tops_px": [
    {"x": 400, "y": 80},
    {"x": 400, "y": 430}
  ],
  "trim_start_ms": 500,
  "trim_end_ms": 3000,
  "ball_color": "red"
}
```

Analysis Result

```
{
  "job_id": "uuid",
  "status": "succeeded",
  "video": {
    "duration_ms": 2500,
    "fps": 30
  },
  "calibration": {
    "mode": "taps",
    "pose": {
      "K": [[fx, 0, cx], [0, fy, cy], [0, 0, 1]],
      "rvec": [rx, ry, rz],
      "tvec": [tx, ty, tz]
    },
    "quality": {
```

```
      "reproj_error_px": 2.1,
      "score": 0.95
    }
  },
  "track": {
    "image_points": [
      {"t_ms": 500, "u": 250, "v": 150, "radius_px": 12, "confidence": 0.85}
    ],
    "inliers": 45,
    "candidates_total": 150,
    "rms_px": 8.3
  },
  "world_trajectory": {
    "points_m": [
      {"t_ms": 500, "x": 15.0, "y": -0.5, "z": 2.0, "confidence": 0.9}
    ],
    "fit": {
      "x0": 15.2, "y0": -0.4, "z0": 2.1,
      "vx": -6.5, "vy": 0.1, "vz": -8.2,
      "bounce_t_ms": 800,
      "rms_m": 0.035
    }
  },
  "events": {
    "bounce": {"t_ms": 800, "x_m": 8.1, "y_m": -0.5},
    "impact": {"t_ms": 1200, "x_m": 0.2, "y_m": -0.1, "z_m": 0.7}
  },
  "lbw": {
    "decision": "out",
    "reason": "Pitched in line, impacted in line, would hit stumps.",
    "checks": {
      "pitching_in_line": true,
      "impact_in_line": true,
      "wickets_hitting": true
    },
    "prediction": {
      "y_at_stumps_m": -0.05,
      "z_at_stumps_m": 0.6,
      "stump_x_m": 0.0,
      "confidence": 0.92
    }
```

```
  },
  "diagnostics": {
    "warnings": [],
    "log_id": "uuid"
  }
}
```

Appendix B: Calibration UI Workflow

The calibration workflow in PocketDRS is designed for simplicity and robustness:

1. **Frame Selection:** User selects any frame from the video where the pitch is clearly visible (typically the beginning of the clip).
2. **Corner Marking:** User taps the four pitch corners in order: striker-end-left, striker-end-right, bowler-end-right, bowler-end-left (clockwise). Visual feedback (circles and lines) confirm each tap.
3. **Quality Feedback:** System computes a homography from the four taps and projects it back to image space. If reprojection error is high, a warning prompts the user to re-calibrate.
4. **Stump Marking (Optional):** If visible, user taps the stump bases and optionally the tops. These refine the camera pose and improve depth accuracy.
5. **Save:** Calibration is saved locally and to Firestore so the user can reuse it for future deliveries from the same ground.
6. **Reuse:** On future videos from the same pitch, user selects a saved calibration rather than re-calibrating.

This workflow balances accuracy (tight constraints from known world points) with usability (minimal user effort). Typical calibration time is 30–60 seconds on a smartphone.